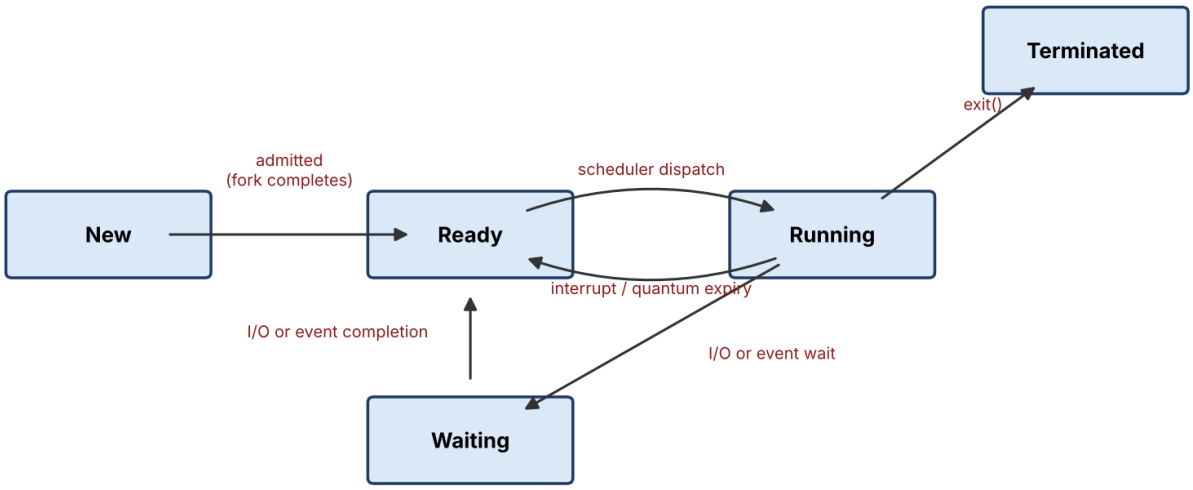
Operating Systems Assignment 2 – Task 1 (Q1)

Processes and Process States – Online Examination Server (CO2, K5 – Evaluate, 6 marks)

Environment: Ubuntu 24.04.5 LTS, gcc 13.3.0, 2 CPUs, 8 GB RAM. Compile command: `gcc -std=c11 -Wall -Wextra -o fork_demo fork_demo.c` (benchmarks: `gcc -O1 -pthread`).

1. Process-state diagram



Transitions: New→Ready when the OS has created the PCB and memory image (end of fork); Ready→Running when the dispatcher picks the process; Running→Ready on timer interrupt or pre-emption; Running→Waiting when the session blocks on disk/network/timer; Waiting→Ready when the event completes; Running→Terminated on exit(). In the exam server, a student session is mostly in Waiting (waiting for keystrokes, auto-save disk writes, network), so it is rarely Running.

2. Process Control Block (PCB)

PCB field	Content	Linux equivalent (task_struct / /proc)
Process ID, parent ID	PID, PPID, user/group IDs	Pid, PPid, Uid in /proc/<pid>/status
Process state	new / ready / running / waiting / terminated	State: R, S, D, Z in /proc/<pid>/status
Program counter & CPU registers	Saved when the process is switched out	saved in kernel stack / thread_struct
CPU-scheduling info	Priority, queue pointers, time slice	nice, policy, /proc/<pid>/sched
Memory-management info	Page-table base, segment limits	mm_struct; VmSize, VmRSS, /proc/<pid>/maps
Accounting info	CPU time used, start time	utime, stime in /proc/<pid>/stat
I/O status info	Open-file table, pending I/O devices	/proc/<pid>/fd, wchan

3. Program: creating children with fork()

Each child models one student session: it prints its PID/PPID, sleeps (Waiting state) and exits; the parent reaps it with wait() so no zombie is left.

```
#define _GNU_SOURCE
#include &lt;stdio.h&gt;
#include &lt;stdlib.h&gt;
#include &lt;unistd.h&gt;
#include &lt;sys/wait.h&gt;
#include &lt;string.h&gt;
int main(int argc, char *argv[]) {
```

```
    int n = (argc &gt; 1) ? atoi(argv[1]) : 5;
    int secs = (argc &gt; 2) ? atoi(argv[2]) : 3;
    printf("Parent PID=%d creating %d children (each lives %d s)\n", getpid(), n, secs);
    fflush(stdout);
    for (int i = 0; i &lt; n; i++) {
        pid_t pid = fork();
        if (pid &lt; 0) { perror("fork"); exit(1); }
        if (pid == 0) {                       /* child: one "student session" */
            printf("  child %d: PID=%d PPID=%d\n", i, getpid(), getppid());
            fflush(stdout);
            sleep(secs);                      /* WAITING state (blocked on timer/I-O) */
            exit(i);                          /* TERMINATED -&gt; zombie until parent waits */
        }
    }
    fflush(stdout);
    int status, done = 0;
    while (done &lt; n) {
        pid_t p = wait(&amp;status);
        if (p &gt; 0) { printf("parent: reaped PID=%d exit=%d\n", p, WEXITSTATUS(status));
done++; }
    }
    return 0;
}
```

Output of `./fork_demo 3 1`

```
Parent PID=276 creating 3 children (each lives 1 s)
  child 0: PID=277 PPID=276
  child 1: PID=278 PPID=276
  child 2: PID=279 PPID=276
parent: reaped PID=277 exit=0
parent: reaped PID=279 exit=2
parent: reaped PID=278 exit=1
```

Observation with ps and /proc (while 5 children are alive)

```
  PID  PPID STAT    VSZ   RSS WCHAN          CMD
  257   255 S      2712   928 hrtimer_nanosl ./fork_demo 5 4
  258   255 S      2712   928 hrtimer_nanosl ./fork_demo 5 4
  259   255 S      2712   928 hrtimer_nanosl ./fork_demo 5 4
  260   255 S      2712   928 hrtimer_nanosl ./fork_demo 5 4
  261   255 S      2712   928 hrtimer_nanosl ./fork_demo 5 4
```

```
Name:■fork_demo
State:■S (sleeping)
Pid:■261
PPid:■255
VmSize:■    2712 kB
VmRSS:■     928 kB
voluntary_ctxt_switches:■1
nonvoluntary_ctxt_switches:■0
```

All five children show STAT **S** (interruptible sleep = Waiting state), PPID 255 (the parent), and WCHAN `hrtimer_nanosleep` (the kernel function they are blocked in). Each child shows ~928 kB RSS and 1 voluntary context switch (it gave up the CPU by sleeping) and 0 involuntary switches. Just after exit and before the parent's wait(), a child would show STAT **Z** (zombie = Terminated, PCB retained).

4. Evaluation: where does the time go with 2,000 active processes?

Measured on this machine (`bench.c`, `spawn2000.c`):

```
fork+exit+wait   : 245.7 us each
pthread create+join: 60.5 us each
process ctx switch (pipe ping-pong, 1 CPU): 1.26 us
thread  ctx switch (pipe ping-pong, 1 CPU): 1.38 us
```

Cost	Measurement / estimate	Evaluation
Process creation	fork+exit+wait ≈ 246 µs each → 2,000 logins ≈ 0.5 s of pure creation work	Login burst is serialised in the kernel (page-table copy, PCB setup). Noticeable but not the main problem.

Cost	Measurement / estimate	Evaluation
Context switch	≈ 1.3 µs direct cost (pipe ping-pong, one CPU). Indirect cost (TLB flush, cache misses) is larger and grows with working-set size.	With only 2 cores, 2,000 ready processes means each gets a time slice only every ~1,000 slices; round-trip scheduling latency is long and sessions appear to “hang”.
Memory per process	Minimal test process: ≈ 0.3 MB RSS; 2,000 running together confirmed (pgrep = 2001). A real exam session (runtime, libraries, buffers) is typically 20–50 MB private memory → 40–100 GB, beyond the 8 GB RAM here.	Memory is the real bottleneck: once RAM is exhausted the system swaps (thrashing) and everything stalls.
I/O wait	Sessions are I/O-bound; most are in Waiting. Auto-save and proctoring video all hit the same disk/network.	Waiting costs no CPU, but contention on shared disk/network queues lengthens each wait; a burst of 2,000 simultaneous saves serialises.

Judgement: A separate process per student is *not* a sound design at this scale. Isolation is good, but the per-process address space, kernel structures and switching overhead are paid 2,000 times for work that is mostly waiting. Sessions share nearly everything (code, exam questions, configuration), so duplicating the address space wastes memory and makes the CPU spend its time switching rather than serving.

5. Design decision

Option	Strengths	Weaknesses
One process per student	Strong isolation; a crash affects one student	Highest memory and creation cost; heavy switches; fails at 2,000
One thread per student	Cheap creation (≈ 60 µs vs 246 µs measured), few-KB stacks, shared data	One bug or crash can kill every session; needs locking for shared data; 2,000 threads still add scheduling overhead
Fixed pool of worker processes (event-driven)	Bounded memory and switching; isolation between workers; scales predictably	More complex design; queueing delay when all workers are busy

Decision: a fixed pool of worker processes, each running many lightweight threads (or an event loop) for sessions, plus a separate small process for proctoring video. Pool size ≈ number of cores × 2–4 (for example 8 workers) keeps context switching and memory bounded; threads/events inside each worker handle the thousands of mostly-waiting sessions cheaply; sessions are distributed across workers so that a crash loses only 1/N of students, and the supervisor restarts the worker while answers are auto-saved to the database. Video proctoring, which is CPU-heavy, is isolated in its own processes so it cannot starve answer-saving.

6. Alternative concept: one thread per session inside one server process

Solution sketch:

```
main():  listen(); loop { c = accept(); pthread_create(&t, &attr(stack=64KB), session, c); pthread_detach(t); }
session(c): loop { read request; update shared exam table under mutex; write response }
```

Aspect	Process per session	Thread per session
Creation time (measured)	≈ 246 µs (fork+exit+wait)	≈ 60 µs (pthread_create+join) → ~4× faster
Context switch (measured)	≈ 1.26 µs direct; plus TLB flush & cache loss on address-space change	≈ 1.38 µs in this virtualised sandbox (equal within noise); on real hardware thread switches avoid the page-table switch/TLB flush, so are generally cheaper
Memory per session	Own page tables + PCB + private data; about 0.3 MB minimum, tens of MB realistic	Own stack (a few KB to 64 KB committed) + small kernel structure; code, heap and exam data shared → 2,000 sessions ≈ tens to hundreds of MB
Communication	IPC (pipes, shared memory, sockets)	Direct shared variables, protected by mutex

Aspect	Process per session	Thread per session
Effect of a crash	Only that student's session dies	A segfault or corrupted heap kills the whole process: all 2,000 sessions lost
Time complexity	Creation O(size of address space) due to page-table copy; switch O(1) + TLB cost	Creation O(1) (no address-space copy); switch O(1)
Space complexity	O(n × M) where M = per-process memory	O(M + n × s) where s = stack size (s ■ M)

Conclusion: threads cut memory and creation cost sharply and are better than 2,000 processes, but they give up fault isolation. The pooled hybrid in Section 5 keeps most of the thread advantage while limiting the blast radius of a crash to one worker.

Note on measurements

Benchmarks ran in a virtualised cloud container with 2 CPUs, so absolute values will differ on other machines; the relative ordering (thread creation much cheaper than fork; memory dominating at 2,000 processes) is the result to note. The summed RSS of the 2,000 test processes counts shared library pages repeatedly, so it overstates true memory use; the 0.3 MB figure is the per-process RSS of a minimal program only.